

Set 3 - Exception Handling

Q1.

Create a Java method `withdrawAmount(int balance, int amount)` that throws an exception if the withdrawal amount is greater than the balance, otherwise subtracts the amount and prints the new balance.

Answer:

```
public class WalletWithdraw {

    public static void withdrawAmount(int balance, int amount) {
        if (amount > balance) {
            throw new IllegalArgumentException("Withdrawal amount exceeds available balance!");
        }
        balance -= amount;
        System.out.println("Withdrawal successful. New balance: Rs. " + balance);
    }

    public static void main(String[] args) {
        withdrawAmount(1000, 400);    // works fine
        withdrawAmount(500, 800);    // throws exception
    }
}
```

/* Sample Output:

Withdrawal successful. New balance: Rs. 600

Exception in thread "main" java.lang.IllegalArgumentException: Withdrawal amount exceeds available bal

*/

Q2.

Simulate a Flipkart-style wallet: write a Java program that asks the user for their wallet balance and purchase amount, then uses try-catch to handle the case where the purchase exceeds the balance by displaying an error message.

Answer:

```
import java.util.Scanner;

public class FlipkartWallet {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your wallet balance: ");
        double balance = sc.nextDouble();

        System.out.print("Enter purchase amount: ");
        double purchase = sc.nextDouble();

        try {
            if (purchase > balance) {
                throw new RuntimeException("Insufficient wallet balance for this purchase.");
            }
            double newBalance = balance - purchase;
            System.out.println("Purchase successful! Remaining balance: Rs. " + newBalance);
        } catch (RuntimeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

```

        System.out.println("Error: " + e.getMessage());
    }

    sc.close();
}

/* Sample Run:
Enter your wallet balance: 500
Enter purchase amount: 750
Error: Insufficient wallet balance for this purchase.
*/

```

Q3.

Build a method `sendMoney(int amount)` for a Paytm-like app that throws a custom `InsufficientFundsException` if the user tries to send more than their current balance. Catch and handle this exception gracefully in `main()`. Hint: Define your own exception class extending `Exception`.

Answer:

```

// Custom exception class
class InsufficientFundsException extends Exception {
    public InsufficientFundsException(String message) {
        super(message);
    }
}

public class PaytmSendMoney {
    private static int balance = 1000;

    public static void sendMoney(int amount) throws InsufficientFundsException {
        if (amount > balance) {
            throw new InsufficientFundsException("Cannot send Rs. " + amount +
                ". Available balance is only Rs. " + balance);
        }
        balance -= amount;
        System.out.println("Rs. " + amount + " sent successfully. Remaining balance: Rs. " + balance);
    }

    public static void main(String[] args) {
        try {
            sendMoney(300);
            sendMoney(2000); // this will throw the custom exception
        } catch (InsufficientFundsException e) {
            System.out.println("Transaction failed: " + e.getMessage());
        }
    }
}

/* Sample Output:
Rs. 300 sent successfully. Remaining balance: Rs. 700
Transaction failed: Cannot send Rs. 2000. Available balance is only Rs. 700
*/

```

Q4.

Write a Java program that reads an array of transaction amounts and processes each one using a for loop and try-catch-finally. Print 'Transaction complete' in the finally block for every transaction, whether it succeeded or failed.

Answer:

```
public class TransactionProcessor {
    public static void main(String[] args) {
        int[] transactions = {500, -200, 1000, 0, 750};
        int balance = 2000;

        for (int i = 0; i < transactions.length; i++) {
            try {
                int amount = transactions[i];
                if (amount <= 0) {
                    throw new IllegalArgumentException("Invalid transaction amount: " + amount);
                }
                if (amount > balance) {
                    throw new IllegalArgumentException("Insufficient balance for transaction: " + amount);
                }
                balance -= amount;
                System.out.println("Transaction " + (i + 1) + " succeeded. New balance: " + balance);
            } catch (IllegalArgumentException e) {
                System.out.println("Transaction " + (i + 1) + " failed: " + e.getMessage());
            } finally {
                System.out.println("Transaction complete");
            }
        }
    }
}
```

/* Sample Output:

```
Transaction 1 succeeded. New balance: 1500
Transaction complete
Transaction 2 failed: Invalid transaction amount: -200
Transaction complete
Transaction 3 succeeded. New balance: 500
Transaction complete
Transaction 4 failed: Invalid transaction amount: 0
Transaction complete
Transaction 5 failed: Insufficient balance for transaction: 750
Transaction complete
*/
```